

STOCK REPORTS LAB

기획·아키텍처 보고서

KRX 장 마감 골든크로스 리포트 MVP

Version 2.0

2026-06-21

MVP 의사결정 완료

1. 보고서 목적

Stock Reports Lab은 KRX 정규시장 종료 후 확정된 일별 데이터를 수집하고, 거래가 활발한 종목에서 Stoch MACD 골든크로스를 탐색하여 시장·업종·종목 단위로 제공하는 장 마감 리포트 서비스다.

이번 버전은 기존 기획서와 스캐너 코드를 바탕으로 다음 목표를 확정한다.

- FinanceDataReader 단일 수집 경로로 MVP 복잡도를 제한한다.
- 실시간 서비스가 아니라 매 거래일 19시에 생성하는 장 마감 리포트로 운영한다.
- 기존 Python 스캐너의 신호 계산 방식을 재현한다.
- Python은 금융 데이터 생산과 분석을, Spring은 조회 API와 사용자 기능을 담당한다.
- 계산된 사실과 AI 서술의 책임을 분리한다.
- 한 종목의 실패가 전체 리포트를 막지 않도록 장애를 종목 단위로 격리한다.

2. MVP 범위

2.1 제공 화면

1. 시장 개요

- KOSPI·KOSDAQ 일봉 차트
- 지수 등락과 스캐너 대상 통계
- AI 시장 요약 한 건

2. 골든크로스 보고서

- 최근 세 거래 일봉 안에 Stoch MACD 골든크로스가 발생한 종목
- 신호 발생일, 현재가, K값, D값, 업종

3. 전체 분석 종목

- 해당 거래일의 분석 종목 200개
- 가격, 거래량, 분석 상태, 최근 신호 조회
- 검색과 정렬

4. 종목 상세

- 1W·1M·3M·1Y 일봉 캔들 차트
- 거래량, MACD, Stoch MACD 시계열
- 골든크로스 발생 마커
- 과거 리포트에서 진입한 경우 해당 거래일까지 차트 범위 제한

5. 업종 현황

- FinanceDataReader의 KRX 설명 목록이 제공하는 업종 사용
- 분석 종목 200개 기준 트리맵과 업종 통계
- 최근 골든크로스 종목 수와 발생 비율

6. 과거 장 마감 리포트

- 서비스 운영 시작일 이후 거래일별 조회
- 사용자에게는 거래일별 최종 활성 리비전만 노출

2.2 MVP 제외 범위

- 실시간 현재가, 호가 및 체결 스트림
- 1분봉·5분봉 등 장중 분봉
- 외국인·기관 수급 데이터
- 네이버 가격 데이터 자동 대체
- 종목별 AI 설명
- 사용자 로그인과 서버 저장 관심종목
- 출시 이전 일별 리포트 소급 생성
- 운영 백업 및 다중 서버 구성의 상세 정책

3. 핵심 용어

장 마감 리포트

해당 거래일의 KRX 정규시장 종가를 기준으로 생성하며, 수집 데이터가 검증된 뒤 공개되는 일별 리포트다. 시간외 거래 가격이나 실시간 가격을 기준으로 하지 않는다.

분석 종목

해당 거래일의 KRX 종목 목록에서 종가가 1,000원 이상인 종목을 거래량 내림차순으로 정렬해 선정한 상위 200개 종목이다. 거래량에 따라 구성 종목은 매 거래일 달라질 수 있다.

업종

FinanceDataReader의 KRX 설명 목록에서 제공하는 거래소 업종 분류다. 별도로 만든 11개 섹터나 테마 분류를 사용하지 않는다.

골든크로스 신호

최근 세 거래 일봉 안에서 Stoch MACD의 K선이 D선을 아래에서 위로 돌파한 사건이다. 달력 기준 최근 3일을 의미하지 않으며 매 수 추천이나 상승 보장을 의미하지 않는다.

4. 데이터 소스와 선정 규칙

4.1 단일 수집 경로

- Python 라이브러리: FinanceDataReader
- 종목 목록과 당일 거래량: StockListing("KRX")
- 종목 설명과 업종: StockListing("KRX-DESC")
- 종목 가격 이력: DataReader("KRX:종목코드", 시작일, 종료일)
- 시장 지수: KOSPI KS11, KOSDAQ KQ11
- 네이버 fallback은 사용하지 않는다.

FinanceDataReader는 데이터 생산자가 아니라 KRX 등의 데이터를 읽는 크롤러다. 특정 시각의 데이터 제공을 보장하는 SLA는 없으므로 재시도와 검증이 필요하다.

4.2 가격 기준

- 종목 차트와 기술지표 계산에는 FDR의 KRX 수정주가 OHLCV를 동일하게 사용한다.
- 원시주가와 수정주가를 혼합하지 않는다.
- 가격 행의 고유 기준은 (종목, 거래일)이다.
- 동일 날짜 재수집은 새 행 추가가 아니라 upsert로 반영한다.

4.3 적재 범위

- 최초 분석 진입 종목: 최근 2년 일봉 적재
- 기존 분석 종목: 매일 최근 10일 재조회 및 upsert
- 분석 대상에서 빠진 종목: 기존 데이터 보존
- 이후 재진입 종목: 보유 이력을 재사용하고 부족한 구간만 보완

5. 배치 실행 흐름

5.1 기본 일정

- 실행 시각: 매일 19:00 Asia/Seoul
- KRX 거래일이 아니면 정상 종료하고 직전 리포트를 유지한다.
- 휴장일을 데이터 실패로 기록하지 않는다.

5.2 처리 순서

19:00 배치 시작

- ├ KRX 거래일 확인
- ├ KRX 종목 목록 조회
- ├ 증가 1,000원 이상 필터
- ├ 거래량 상위 200개 선정
- ├ 종목별 순차 수집 및 upsert
- ├ 기술지표와 신호 계산
- ├ 최초 리비전 공개
- ├ 실패 종목만 재시도
- ├ 변경분을 최종 리비전으로 1회 공개
- └ AI 시장 요약 비동기 생성

5.3 요청 제어

- 200개 종목을 동시에 요청하지 않고 한 종목씩 순차 처리한다.
- 종목별 요청 제한시간은 30초다.
- 5개 종목이 연속으로 시간 초과하면 공급자 장애로 보고 현재 회차를 중단한다.
- 실패·미처리 종목은 10분 간격으로 최대 3회 추가 재시도한다.
- 한 종목의 실패는 다른 종목 처리를 중단시키지 않는다.

5.4 게시 정책

- 첫 수집 회차가 끝나면 정상 종목 결과를 최초 리비전으로 공개한다.
- 재시도 성공 때마다 새 리비전을 만들지 않는다.

- 모든 데이터 재시도가 끝난 뒤 변경분을 모아 최종 리비전 한 번만 추가한다.
- 종목 목록 자체를 가져오지 못하면 당일 분석 대상을 정할 수 없으므로 전체 리포트는 생성 지연 상태가 되고 직전 리포트를 유지한다.

6. 종목 처리 상태

상태	의미	전체 리포트 영향
신호 발견	최근 세 거래 일봉 안에 골든크로스 존재	정상 공개
신호 없음	판정 완료, 최근 신호 없음	정상 공개
분석 데이터 부족	일봉 110개 미만	해당 종목 지표 미제공
데이터 준비 중	최신 일봉 재시도 진행 중	다른 종목 정상 공개
데이터 갱신 실패	최대 재시도 안에 최신 일봉 확보 실패	해당 종목만 실패 표시
당일 거래 없음	새 일봉은 없지만 이전 데이터로 분석 가능	마지막 실제 거래일 표시
분석 실패	데이터는 있으나 결정론적 계산 실패	해당 종목만 실패 표시

당일 거래 없음 상태에서는 이전 분석을 조회할 수 있지만 해당 거래일의 신규 골든크로스로 발행하지 않는다.

7. 기술지표와 신호 계산

7.1 계산 책임

- 모든 가격 파생값, 기술지표, 신호, 순위와 업종 집계는 Python 결정론적 코드가 계산한다.
- Spring과 프론트엔드는 기술지표를 재계산하지 않는다.
- AI는 계산된 결과를 설명할 뿐 수치나 원인을 새로 만들지 않는다.

7.2 기존 스캐너 재현

- MACD: fast 12, slow 26, signal 9
- Stoch MACD: K 14, D 3, smooth K 3
- MACD 선 자체를 Stochastic의 high·low·close 입력으로 사용한다.
- 최근 세 거래 일봉의 K·D 교차를 검사한다.
- 신호 식별자: (종목, 신호 종류, 발생일, 계산 버전)
- 동일 신호는 한 번만 저장하고 세 거래 일봉 동안 최근 신호로 재사용한다.

7.3 계산 버전

- 최초 계산법은 stoch-macd-v1로 고정한다.
- 계산 버전은 사용자 기능이 아니라 재현성과 개발 추적을 위한 메타데이터다.
- 계산법 변경 시 기존 값을 덮어쓰지 않고 v2 결과를 추가한다.
- Spring은 계산식 내용을 모르며 리포트가 참조하는 버전의 결과만 조회한다.

8. 업종 분석 규칙

- 대상 범위: 해당 거래일 분석 종목 200개
- 트리맵 면적: 업종별 시가총액 합계
- 트리맵 색상: 업종 종목의 일간 등락률 단순 평균
- 단순 평균을 사용해 골든크로스 스캐너의 개별 종목 중심 관점과 맞춘다.

- 업종별 포함 종목 수와 최근 골든크로스 종목 수를 함께 표시한다.
- 최근 골든크로스 비율은 최근 세 거래 일봉 안에 신호가 발생한 종목 수를 신호 판정 완료 종목 수로 나눈 값이다.
- 분석 데이터 부족, 데이터 갱신 실패, 당일 거래 없음 종목은 비율의 분모에서 제외하고 제외 수를 표시한다.

9. AI 시장 요약

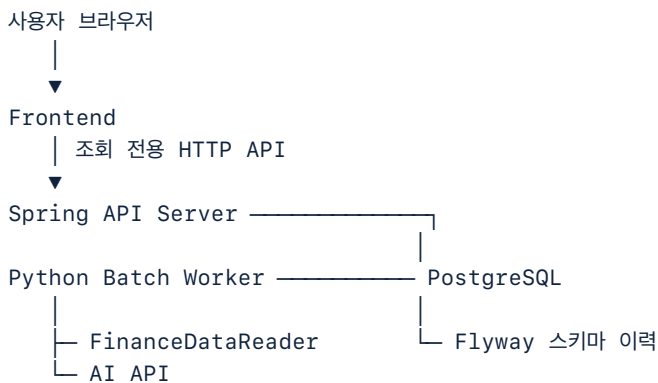
9.1 범위

- AI 호출은 거래일별 시장 전체 요약 한 건으로 제한한다.
- 종목별 AI 설명은 생성하지 않는다.
- 입력은 KOSPI·KOSDAQ 지수 통계와 거래량 상위 200개 스캐너 통계를 분리하여 제공한다.
- 입력 예: 지수 등락, 상승·하락 종목 수, 최근 골든크로스 수, 주요 업종 통계.

9.2 실행과 실패

- 종목 데이터 재시도가 모두 끝난 뒤 AI를 한 번 생성한다.
- AI 실패 시 1분·5분·15분 간격으로 최대 3회 재시도한다.
- 재시도 중에는 분석 중으로 표시한다.
- 모든 재시도 실패 시 분석 생성 지연으로 표시한다.
- AI 실패와 관계없이 검증된 가격·차트·지표·신호는 제공한다.
- AI 상태와 내용 변경만으로 금융 데이터 리비전을 추가하지 않는다.

10. 시스템 아키텍처



단일 서버 / Docker Compose

10.1 Frontend

- 시장 개요, 골든크로스 보고서, 전체 종목, 종목 상세, 업종, 과거 리포트 화면을 제공한다.
- 관심종목은 브라우저 로컬 저장소에만 보관한다.
- 서버 계정과 기기 간 동기화는 제공하지 않는다.

10.2 Spring API Server

- 공개 API는 조회 전용 GET으로 제한한다.
- 거래일별 활성 리비전과 연결된 결과만 제공한다.
- 검색, 정렬, 날짜 선택, 관심종목 화면 지원 등 애플리케이션 로직을 담당한다.
- 지표, 신호, 순위 및 업종 통계를 재계산하지 않는다.
- DB 구조 변경은 Spring의 Flyway가 단독 관리한다.

10.3 Python Batch Worker

- 종목 선정, 데이터 수집, 검증, upsert, 기술지표, 신호, 업종 집계와 AI 입력 생성을 담당한다.
- Spring 내부 API를 호출하지 않고 PostgreSQL에 직접 기록한다.
- Flyway가 만든 승인된 테이블만 사용하며 실행 중 스키마를 변경하지 않는다.
- 독립 프로세스로 실행되어 Spring 장애가 배치 실행에 전파되지 않게 한다.

10.4 PostgreSQL

- Spring과 Python이 하나의 DB를 공유한다.
- Python 소유 영역은 금융 원천·분석 결과이며 Spring은 읽기 전용으로 사용한다.
- Spring 소유 영역은 애플리케이션 데이터다.
- 사용자에게는 최신 적재 행이 아니라 최신 게시 완료 리비전을 제공한다.

11. 데이터 모델

핵심 업무 테이블은 8개다.

테이블	주요 책임
stock	종목코드, 현재 종목명, 시장, 현재 업종
daily_price	종목별 수정주가 OHLCV와 거래일
daily_indicator	계산 버전별 일자 MACD, Stoch K·D
market_index_price	KOSPI·KOSDAQ 지수 일봉
report_revision	거래일, 리비전, 활성 상태, 커버리지, AI 상태·요약
stock_analysis	리비전별 분석 대상, 상태, 종목명·시장·업종 스냅샷
signal_event	종목별 골든크로스 발생 사건
industry_analysis	리비전별 업종 규모, 평균 등락률, 신호 수·비율

11.1 주요 고유 조건

- daily_price: (stock_id, trade_date)
- daily_indicator: (stock_id, trade_date, calculation_version)
- stock_analysis: (report_revision_id, stock_id)
- signal_event: (stock_id, signal_type, cross_date, calculation_version)
- market_index_price: (index_code, trade_date)
- industry_analysis: (report_revision_id, industry_name)

11.2 스냅샷 원칙

- stock에는 현재 메타데이터를 저장한다.
- 과거 리포트 맥락 보존을 위해 stock_analysis에는 당시 종목명·시장·업종을 함께 저장한다.
- 가격 일봉은 리비전마다 복제하지 않는다.
- 업종과 분석 결과만 리비전에 연결한다.

12. 리비전과 과거 조회

- 같은 거래일 결과가 데이터 재시도나 정정으로 달라지면 새 리비전을 만든다.
- 이전 리비전은 비교·원인 추적·복구를 위해 보존한다.
- 사용자에게는 거래일별 활성 리비전 하나만 노출한다.
- 미완료 리비전은 활성화하지 않는다.
- 과거 리포트는 서비스 운영 시작 거래일부터 제공한다.
- 초기 2년 가격에서 계산한 과거 신호는 종목 차트 참고 마커로 표시할 수 있으나 당시 발행 리포트로 취급하지 않는다.

13. 조회 API 개념

정확한 URL과 응답 스키마는 구현 계획에서 확정하되 다음 조회 경계를 유지한다.

- 최신 장 마감 리포트
- 거래일별 과거 리포트
- 리비전별 골든크로스 종목 목록
- 리비전별 전체 분석 종목 목록
- 종목 기본정보와 상태
- 기준일까지의 종목 가격·지표·신호 시계열
- 리비전별 업종 분석
- KOSPI·KOSDAQ 지수 시계열

전체 분석 종목은 200개이므로 MVP 목록 API는 복잡한 서버 페이지네이션 없이 한 번에 제공할 수 있다. 차트는 요청한 종목과 기간만 반환한다.

14. 사용자 표시 원칙

- 모든 화면에 데이터 기준 거래일을 표시한다.
- 분석 범위가 KRX 전체가 아니라 종가 1,000원 이상·거래량 상위 200개임을 명시한다.
- 업종 화면에는 거래량 상위 200개 기준을 표시한다.
- 과거 리포트 상세 차트는 선택한 리포트 거래일 이후 데이터를 섞지 않는다.
- 신호 발생일과 보고서 표시일을 구분한다.
- 골든크로스를 매수 추천이나 상승 보장으로 표현하지 않는다.

15. 검증과 수용 기준

데이터 수집

- 국내 종목 가격 호출은 모두 KRX: 소스를 명시한다.
- 동일 종목·거래일 재실행 시 가격 중복 행이 생기지 않는다.
- 신규 분석 종목은 지표 계산에 필요한 과거 이력을 확보한다.
- 휴장일은 실패 리포트를 만들지 않는다.

분석

- 고정된 가격 fixture에서 MACD와 Stoch MACD 결과가 반복 실행마다 동일하다.
- K선이 D선을 아래에서 위로 돌파한 날짜만 신호로 저장한다.
- 동일 신호를 여러 보고서가 참조해도 signal_event는 한 건이다.
- 110개 미만 일봉은 분석 데이터 부족으로 구분한다.
- 계산 버전이 다른 지표가 한 리포트에 섞이지 않는다.

장애 격리

- 한 종목 수집 실패가 다른 종목 처리를 멈추지 않는다.
- 데이터 준비 중과 데이터 갱신 실패가 구분된다.
- 종목 목록 실패 시 직전 활성 리포트를 유지한다.
- AI 실패 시에도 계산 결과와 차트를 제공한다.

API와 화면

- Spring은 활성 리비전만 기본 조회한다.
- 과거 거래일 조회는 해당 날짜의 활성 리비전을 반환한다.
- 상세 캔들 차트와 지표 시계열의 마지막 날짜가 요청 기준일을 넘지 않는다.
- 업종 통계는 신호 판정 완료 종목만 비율의 분모로 사용한다.

16. 구현 전제와 후속 결정

다음 항목은 핵심 아키텍처를 변경하지 않으므로 구현 계획에서 확정한다.

- Frontend 프레임워크와 차트 라이브러리의 정확한 선택
- Spring Boot 및 JVM 언어·버전
- AI 제공자, 모델과 프롬프트 스키마
- API URL, DTO와 오류 응답 형식
- 배포 자동화, 모니터링, 외부 백업과 보존 기간
- 실제 성능 측정 후 제한 병렬 처리 도입 여부

17. 최종 결정 요약

MVP는 실시간 종합 주식 플랫폼이 아니라, FDR의 KRX 일봉을 기반으로 거래량 상위 200개를 안정적으로 스캔하는 장 마감 리포트다. Python이 금융 데이터와 분석 결과를 생산하고 Spring이 게시 완료된 결과를 조회 전용 API로 제공한다. PostgreSQL의 활성 리비전이 사용자에게 보이는 단일 기준이며, 한 종목의 실패와 AI 실패는 정상 결과로부터 격리한다.

이 구조는 현재 스캐너의 장점을 유지하면서 파일 누적 방식의 중복·부분 실패·추적 한계를 제거하고, 이후 종목 수 확대나 추가 신호 도입이 가능하도록 계산 버전과 리포트 리비전을 분리한다.